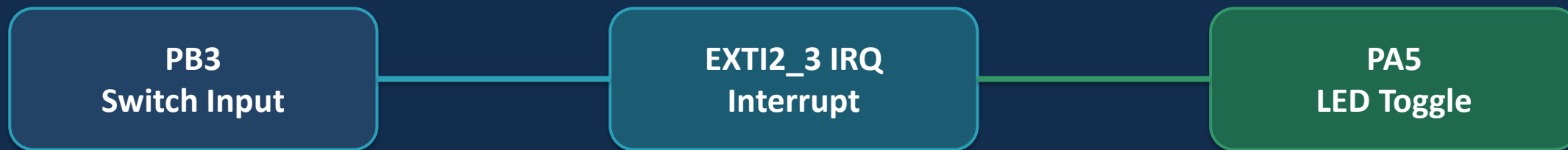


GPIO 입력 방식: Polling → Interrupt 전환

STM32G0 · PB3 Switch · PA5 LED · EXTI/NVIC/HAL Callback



기본 GPIO 예제를 인터럽트 기반 펌웨어 구조로 변경

입력 이벤트 처리의 두 가지 기본 방식

Polling은 CPU가 묻고, Interrupt는 하드웨어가 CPU를 호출합니다.

Polling : “스위치 눌렀니?”를 계속 확인



CPU가 입력 레지스터를 계속 읽으므로
구조가 단순하지만 대기 시간에도 CPU를 사용

Interrupt : “눌리면 나를 깨워줘!”

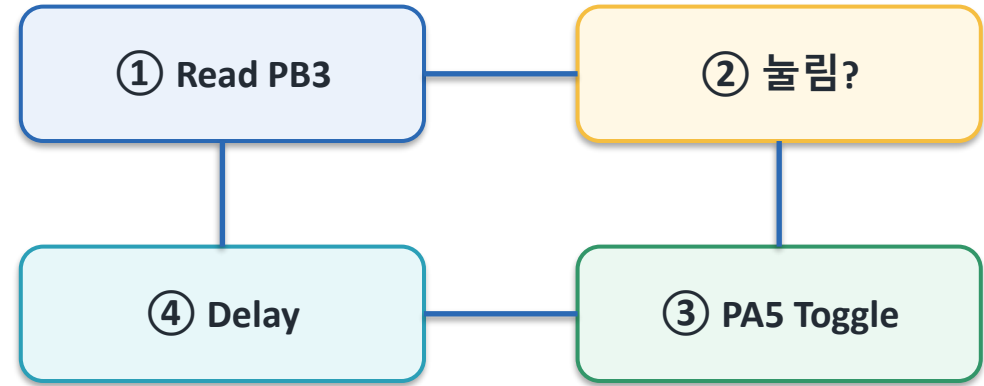


평소에는 다른 일을 하다가 이벤트가 발생하면
ISR/Callback을 통해 필요한 처리만 수행

Polling 방식 – 기존 STM32 코드

while(1) 안에서 HAL_GPIO_ReadPin()을 반복 호출합니다.

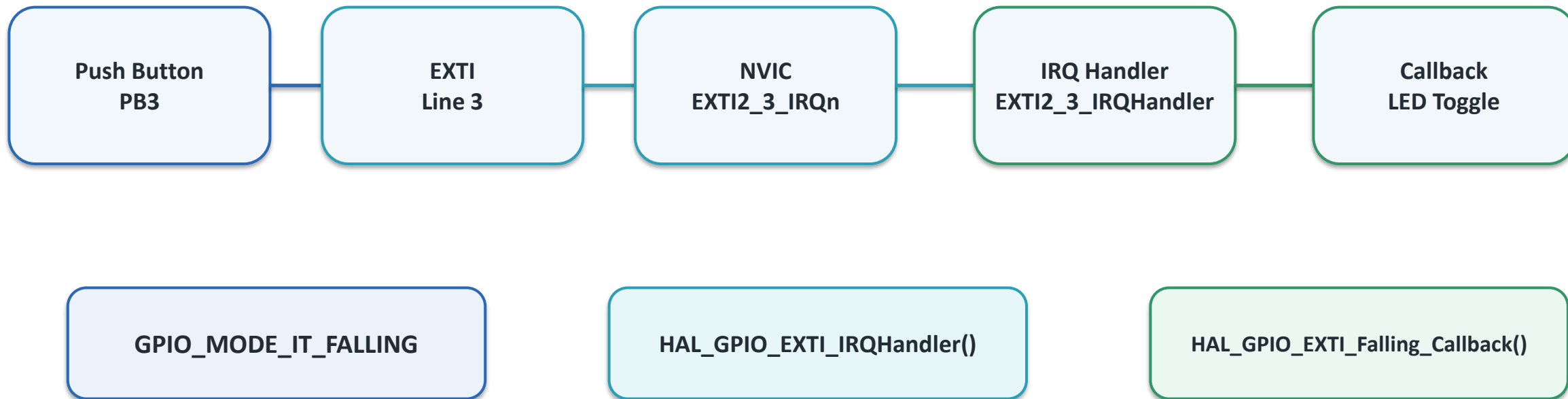
```
while (1)
{
    if (HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_3)
        == GPIO_PIN_RESET)
    {
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
        HAL_Delay(200);
    }
}
```



핵심: 입력이 없어도 CPU는 계속 PB3 상태를 확인합니다.
이 예제의 200 ms Delay는 간단한 채터링 억제 효과도 있지만,
그 동안 CPU가 다른 polling 작업을 즉시 처리하기 어렵습니다.

Interrupt 방식 – STM32G0의 처리 경로

PB3의 Falling Edge → EXTI Line 3 → NVIC → IRQ Handler → HAL Callback



STM32G0 HAL에서는 EXTI interrupt mode 사용 시 IRQ Handler에서 HAL_GPIO_EXTI_IRQHandler()를 호출하고, Rising/Falling callback에서 사용자 동작을 구현하는 구조가 핵심입니다.

코드로 비교하는 Polling ↔ Interrupt

같은 PB3 스위치와 PA5 LED를 사용하지만 '이벤트를 발견하는 주체'가 달라집니다.

Polling

```
GPIO_InitStruct.Mode = GPIO_MODE_INPUT;  
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);  
  
while (1) {  
    if (HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_3)  
        == GPIO_PIN_RESET) {  
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);  
    }  
}
```

CPU가 이벤트를 '찾는다'

Interrupt

```
GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;  
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);  
  
HAL_NVIC_EnableIRQ(EXTI2_3_IRQn);  
  
void HAL_GPIO_EXTI_Falling_Callback(uint16_t pin)  
{  
    if (pin == GPIO_PIN_3)  
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);  
}
```

하드웨어가 이벤트를 '알린다'

언제 Polling, 언제 Interrupt?

둘 중 하나가 항상 우월한 것이 아니라 요구사항에 따라 선택합니다.

Polling이 적합한 경우

구조를 최대한 단순하게 만들고 싶을 때
이벤트가 매우 자주 발생해 지속적으로 상태를 읽어야 할 때
정해진 주기의 제어 루프에서 여러 입력을 함께 검사할 때
응답시간 요구가 polling 주기보다 느슨할 때

Interrupt가 적합한 경우

버튼처럼 이벤트가 드물고 비동기적으로 발생할 때
이벤트를 빠르게 감지해야 할 때
CPU가 다른 작업을 수행하거나 저전력 모드에 있어야 할 때
여러 peripheral 이벤트를 효율적으로 처리해야 할 때

실무 패턴: ISR에서는 짧게 flag만 설정하고, 시간이 오래 걸리는 처리는 main loop/task에서 수행

버튼 Interrupt에서 꼭 설명할 것 — Debouncing

기계식 스위치는 한 번 눌러도 짧은 시간 동안 여러 edge가 발생할 수 있습니다.

이상적인 입력



실제 스위치



문제
한 번 눌러도 여러 IRQ 가능

방법 1
HAL_GetTick() 시간 필터

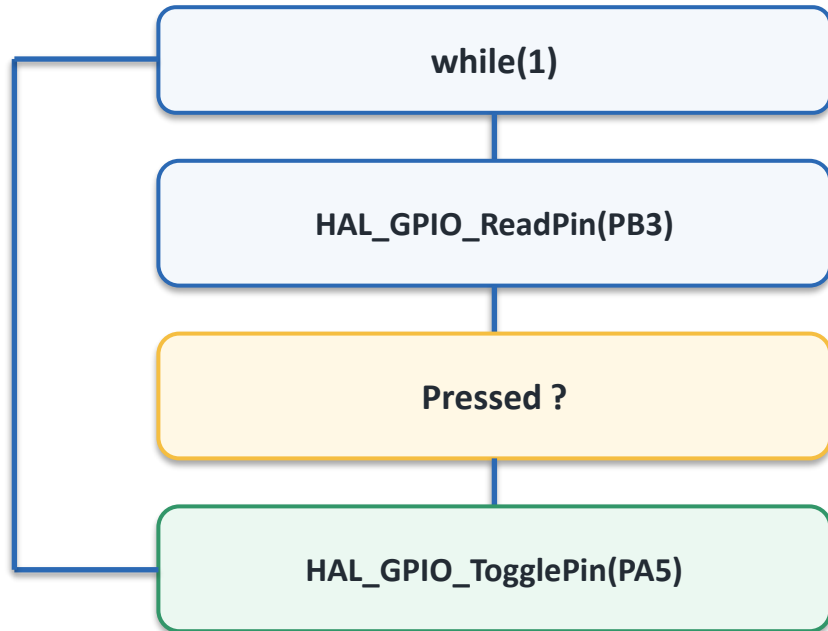
방법 2
Timer 기반 debounce

강의 포인트: ISR 안에서 HAL_Delay()로 오래 기다리기보다, timestamp/flag/timer를 이용해 짧게 처리하는 구조가 일반적으로 더 안전합니다.

두 방식의 Flow Chart 비교

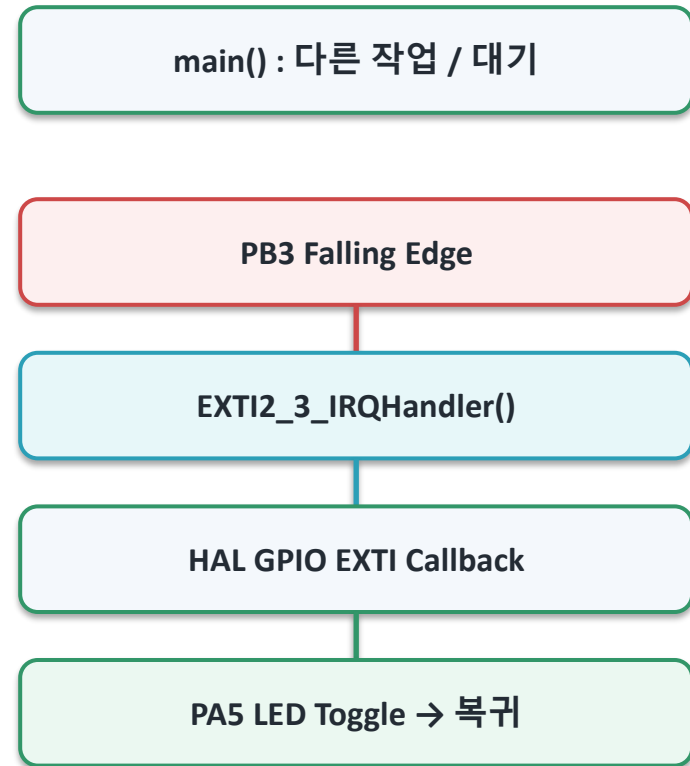
동일한 기능을 서로 다른 제어 흐름으로 구현합니다.

Polling

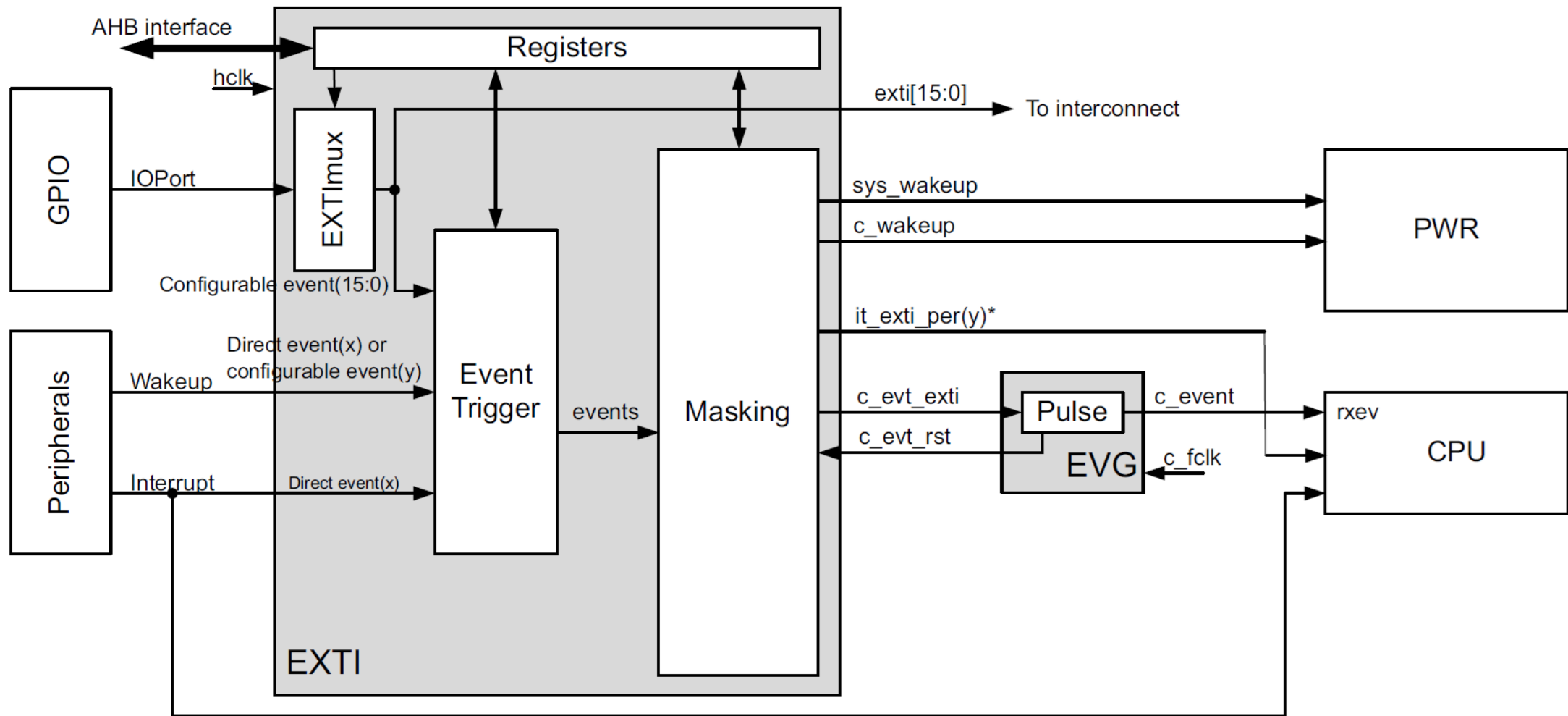


주기적으로 검사

Interrupt

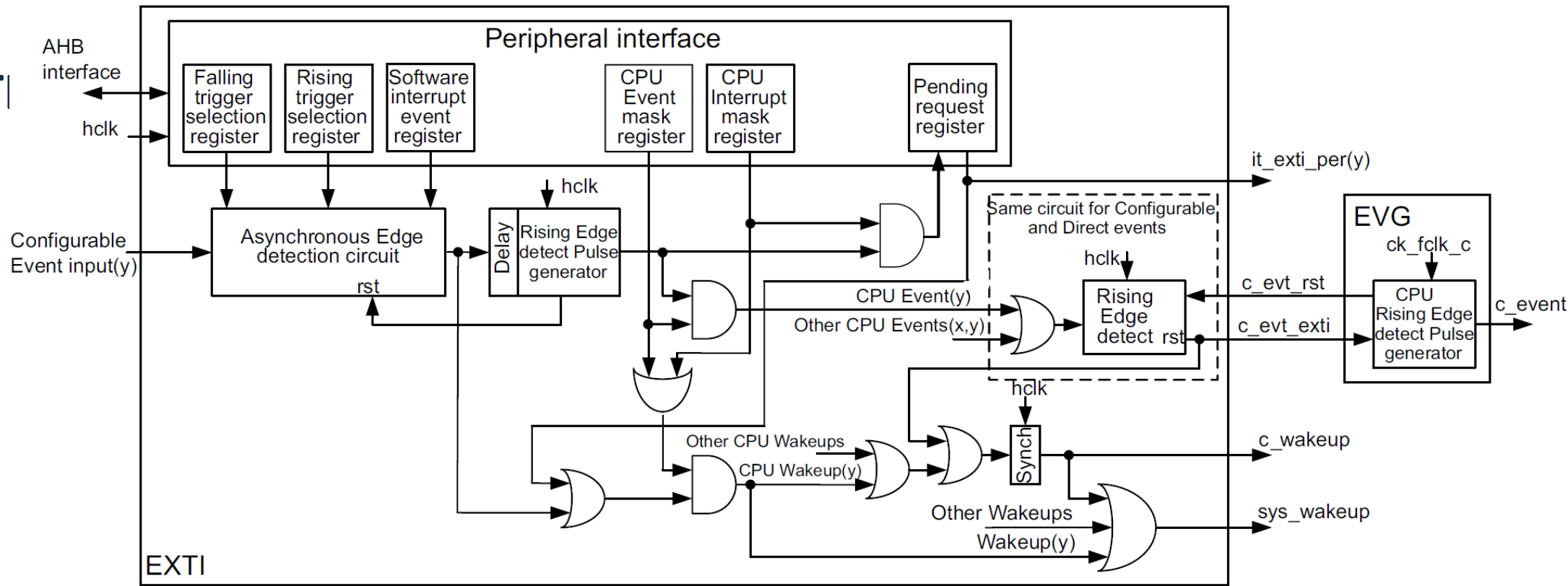


이벤트가 실행 흐름을 잠시 선점



* it_exti_per(y) are only available for configurable events (y)

예제 EXTI



MS46537V1

EXTI : 외부에서 발생하는 Interrupt, 스위치 눌러 수행하는 동작 등
STM32는 위와 같이 처리함.

- 1) Input Line을 통해 인터럽트가 발생하면, rising 또는 falling을 감지하여 OR gate를 통하여 신호 전달
- 2) Software interrupt event, rising, falling trigger selection register 의 신호를 OR 게이트로 전달하고, 전달된 신호는 Interrupt mask register와 AND gate 연결
- 3) Interrupt mask register의 경우

Table 62. EXTI signal overview

Signal name	I/O	Description
AHB interface	I/O	EXTI register bus interface. When one event is configured to allow security, the AHB interface support secure accesses
hclk	I	AHB bus clock and EXTI system clock
Configurable event(y)	I	Asynchronous wake-up events from peripherals that do not have an associated interrupt and flag in the peripheral
Direct event(x)	I	Synchronous and asynchronous wake-up events from peripherals having an associated interrupt and flag in the peripheral
IOPort(n)	I	GPIO ports[15:0]
exti[15:0]	O	EXTI output port to trigger other IPs
it_exti_per (y)	O	Interrupts to the CPU associated with configurable event (y)
c_evt_exti	O	High-level sensitive event output for CPU synchronous to hclk
c_evt_rst	I	Asynchronous reset input to clear c_evt_exti
sys_wakeup	O	Asynchronous system wake-up request to PWR for ck_sys and hclk
c_wakeup	O	Wake-up request to PWR for CPU, synchronous to hclk

기존 polling 구조 입력에서 interrupt 방식 입력으로 전환

포트/핀은 그대로 유지하고, 입력 감지 방식만 폴링에서 인터럽트로 변경합니다.

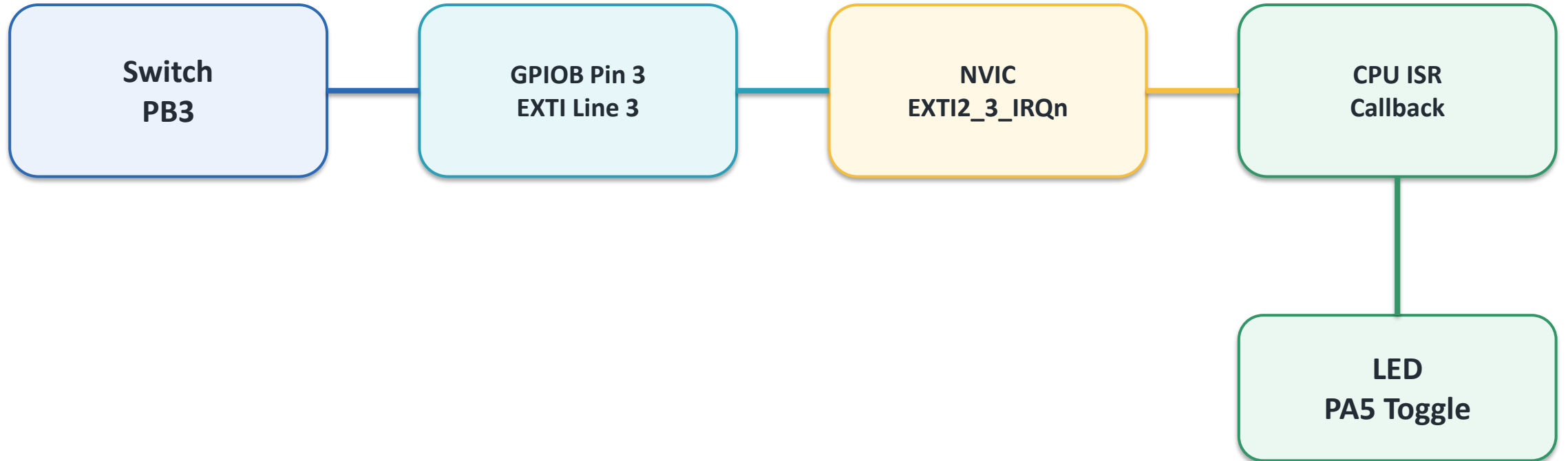
기존 Polling 구조
while(1)에서 PB3 상태를 계속 읽음
눌림 감지 시 PA5 LED Toggle

변경 Interrupt 구조
PB3 Falling Edge가 EXTI interrupt 발생
Callback에서 PA5 LED Toggle

CPU가 버튼 입력을 계속 감시하지 않아도 됨
main loop는 idle, sleep, 다른 task 수행 가능
입력 이벤트가 발생했을 때만 ISR/Callback 실행
단, ISR 내부에서는 HAL_Delay() 같은 blocking delay를 사용하지 않음

Interrupt 기반 Architecture

스위치 입력은 EXTI Line 3을 통해 NVIC로 전달되고, HAL Callback에서 LED를 제어합니다.



핵심 흐름: PB3가 High → Low로 변하는 순간, EXTI가 인터럽트를 만들고 CPU가 지정된 ISR을 실행한다.

GPIO 설정 변경 포인트

LED 출력 설정은 유지하고, 스위치 입력을 EXTI Falling Edge 모드로 바꿉니다.

```
/* PA5: LED Output */
GPIO_InitStruct.Pin   = GPIO_PIN_5;
GPIO_InitStruct.Mode  = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull  = GPIO_PULLUP;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
```

PA5는 Push-Pull 출력
LED를 직접 ON/OFF 제어

```
/* PB3: Switch Interrupt Input */
GPIO_InitStruct.Pin   = GPIO_PIN_3;
GPIO_InitStruct.Mode  = GPIO_MODE_IT_FALLING;
GPIO_InitStruct.Pull  = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
```

PB3는 Falling Edge Interrupt
스위치 눌림 순간 IRQ 발생

NVIC와 IRQ Handler

EXTI Line 3은 STM32G0에서 EXTI2_3_IRQn 인터럽트 벡터로 처리합니다.

```
HAL_NVIC_SetPriority(EXTI2_3_IRQn, 2, 0);  
HAL_NVIC_EnableIRQ(EXTI2_3_IRQn);
```

```
void EXTI2_3_IRQHandler(void)  
{  
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_3);  
}
```

Priority 설정
IRQ 우선순위 지정

IRQ Enable
인터럽트 허용

HAL Handler
Pending bit 처리 + Callback 호출

CubeIDE 프로젝트에서는 EXTI2_3_IRQHandler()가 보통 stm32g0xx_it.c에 위치합니다. main.c에 중복 정의하지 않도록 주의합니다.

workspace_2.2.0 - GPIO_IOToggle/Application/Startup/startup_stm32g0b1retx.s - STM32CubeIDE

File Edit Source Refactor Navigate Search Project Run Window Help

Project Explorer

DMA_FLASHToRAM (in STM32CubeIDE)

FLASH_EraseProgram (in STM32CubeIDE)

GPIO_EXTI (in STM32CubeIDE)

GPIO_IOToggle (in STM32CubeIDE)

Binaries

Includes

C:/ST/STM32CubeIDE_2.2.0/STM32Cub

C:/ST/STM32CubeIDE_2.2.0/STM32Cub

C:/ST/STM32CubeIDE_2.2.0/STM32Cub

C:/ST/STM32CubeIDE_2.2.0/STM32Cub

C:/Temp/Development/ngv/ngv/STM3

C:/Temp/Development/ngv/ngv/STM3

C:/Temp/Development/ngv/ngv/STM3

C:/Temp/Development/ngv/ngv/STM3

C:/Temp/Development/ngv/ngv/STM3

C:/Temp/Development/ngv/ngv/STM3

E:/files/STM32CubeG0-master/STM32C

E:/files/STM32CubeG0-master/STM32C

E:/files/STM32CubeG0-master/STM32C

E:/files/STM32CubeG0-master/STM32C

Application

Startup

startup_stm32g0b1retx.s

User

main.c

stm32g0xx_hal_msp.c

stm32g0xx_it.c

syscalls.c

systemem.c

Debug

Doc

Drivers

GPIO_IOToggle.ioc

GPIO_IOToggle.launch

STM32G0B1RETX_FLASH.ld

GPIONTest2

GPIONTest5

RTC_LowPower_STANDBY (in STM32CubeID

Templates (in STM32CubeIDE)

testnri1

stm3210xx_h...

stm321073xx.h

stm3210xx_nu...

main.c

startup_stm...

stm32g0b1xx.h

131 g_pfnVectors:

132 .word _estack

133 .word Reset_Handler

134 .word NMI_Handler

135 .word HardFault_Handler

136 .word 0

137 .word 0

138 .word 0

139 .word 0

140 .word 0

141 .word 0

142 .word 0

143 .word SVC_Handler

144 .word 0

145 .word 0

146 .word PendSV_Handler

147 .word SysTick_Handler

148 .word WWDG_IRQHandler /* Window WatchDog */

149 .word PVD_VDDIO2_IRQHandler /* PVD through EXTI Line detect */

150 .word RTC_TAMP_IRQHandler /* RTC through the EXTI line */

151 .word FLASH_IRQHandler /* FLASH */

152 .word RCC_CRs_IRQHandler /* RCC & CRS */

153 .word EXTI0_1_IRQHandler /* EXTI Line 0 and 1 */

154 .word EXTI2_3_IRQHandler /* EXTI Line 2 and 3 */

155 .word EXTI4_15_IRQHandler /* EXTI Line 4 to 15 */

156 .word USB_UCPD1_2_IRQHandler /* USB, UCPD1, UCPD2 */

157 .word DMA1_Channel1_IRQHandler /* DMA1 Channel 1 */

158 .word DMA1_Channel12_3_IRQHandler /* DMA1 Channel 2 and Channel 3 */

Outline

Build Targets

Reset_Handler

CopyDataInit

LoopCopyDataInit

FillZeroBss

LoopFillZeroBss

LoopForever

Default_Handler

g_pfnVectors

Problems

0 errors, 9 warnings, 0 others

Description

Warnings (9 items)

Tasks Console Properties

<terminated> GPIO_IOToggle [STM32 C/C++ Application] ST-LINK (ST-LINK GDB server) (Terminated 2026. 9. 9. 오전 4:59: Download verified successfully

Build An...

Static St...

Cycloma...

Problem...

Debug

<terminated>GPIO_IOToggle [STM32 C/C++ Application]

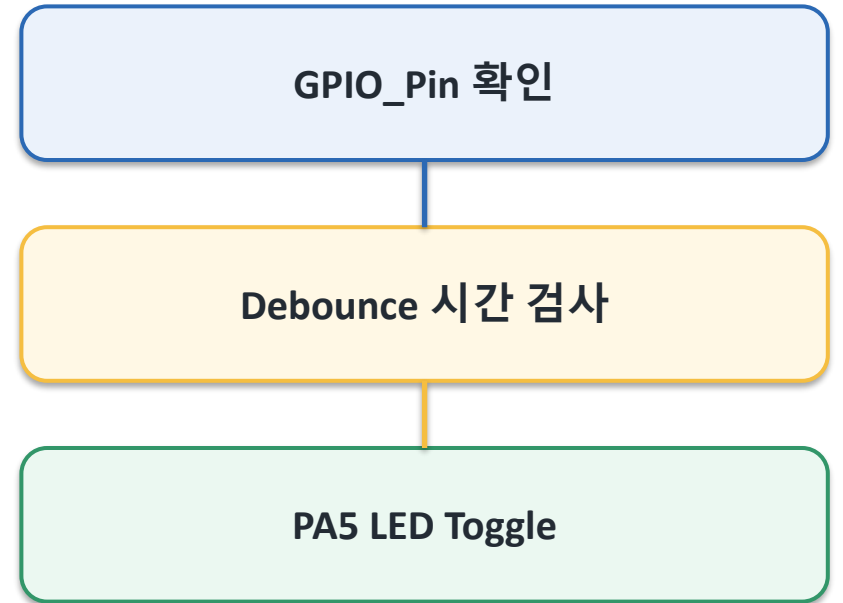
<terminated, exit value: 0>arm-none-eabi-gdb (15.2.90.20241229)

<terminated, exit value: 0>ST-LINK (ST-LINK GDB server)

Callback에서 LED Toggle

인터럽트 발생 후 HAL이 사용자 Callback을 호출합니다.

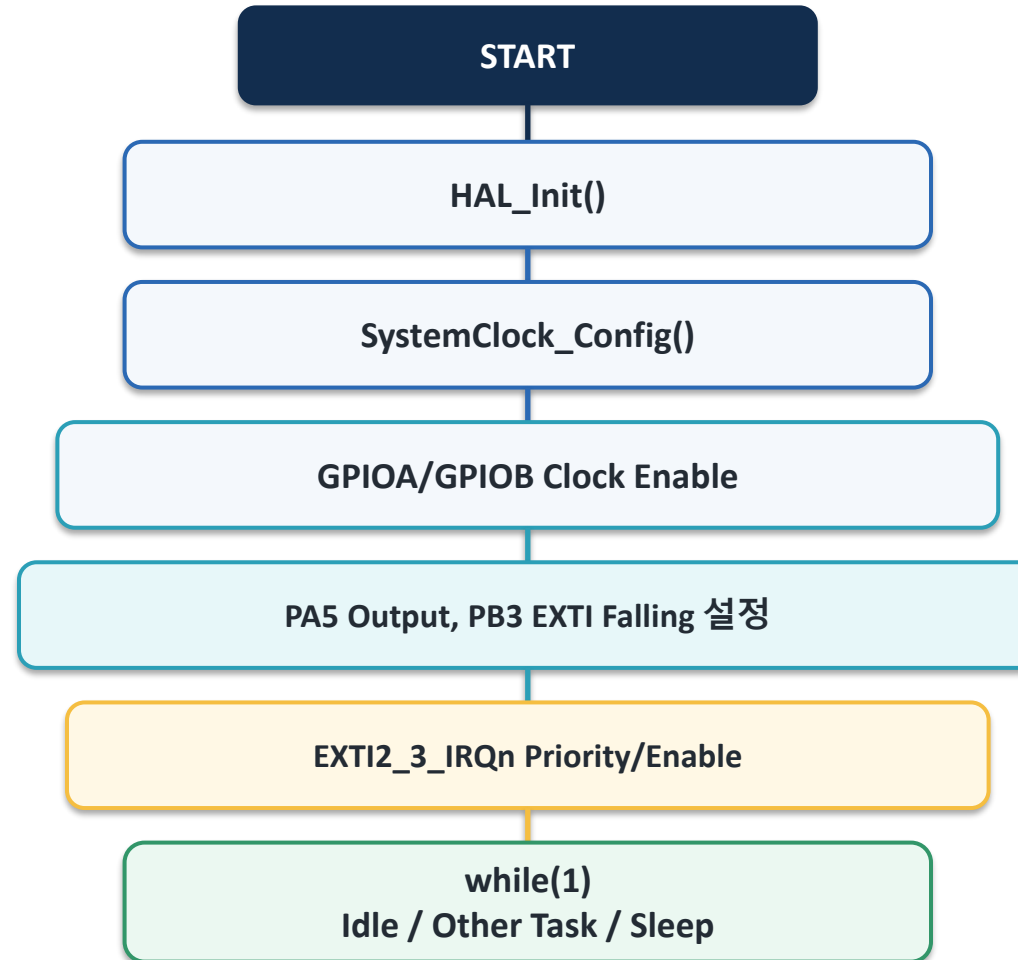
```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == GPIO_PIN_3)
    {
        uint32_t now = HAL_GetTick();
        if ((now - lastButtonTick) >= 200U)
        {
            lastButtonTick = now;
            HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
        }
    }
}
```



중요: ISR/Callback 내부에서는 HAL_Delay() 대신 HAL_GetTick() 기반의 비차단 debounce를 사용합니다.

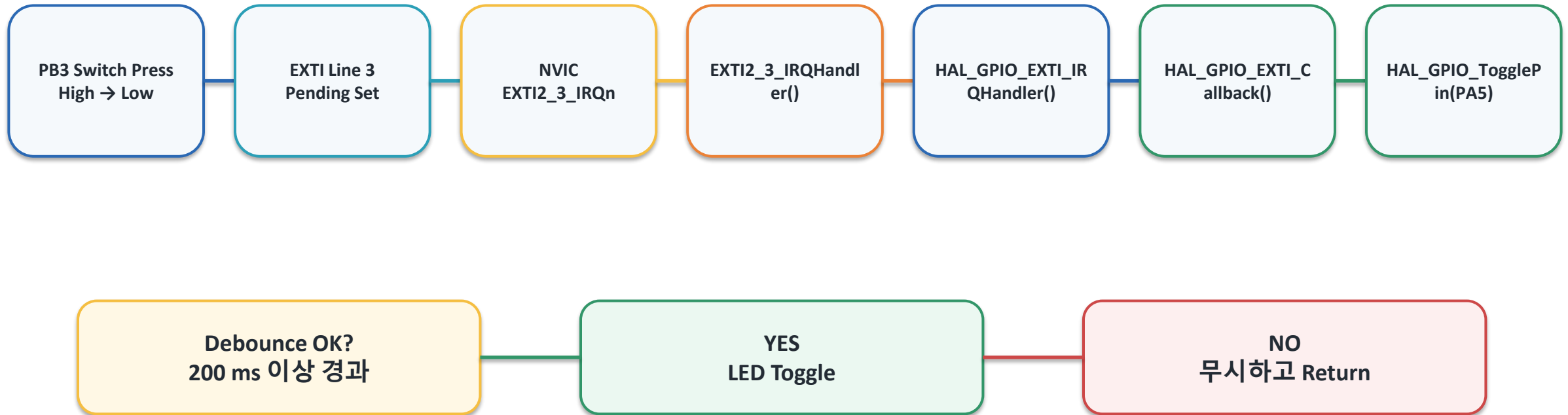
main() 프로그램 Flow Chart

초기화 이후 main loop는 입력을 폴링하지 않고 대기 상태로 유지됩니다.



Interrupt Event Flow Chart

스위치가 눌린 순간부터 LED가 토글되기까지의 이벤트 흐름입니다.



기계식 스위치는 bounce가 발생하므로, 인터럽트 예제에서도 debounce 처리가 필요합니다.

주요 함수 기능 정리

강의 시 함수별 책임을 분리해서 설명하면 구조가 명확해집니다.

HAL_Init()	HAL 라이브러리와 SysTick time base 초기화
SystemClock_Config()	HSI + PLL 기반 시스템 클럭 설정
__HAL_RCC_GPIOx_CLK_ENABLE()	GPIO 설정 레지스터 접근을 위해 포트 클럭 활성화
HAL_GPIO_Init()	PA5 출력, PB3 EXTI 입력 모드 설정
HAL_NVIC_SetPriority()	EXTI2_3_IRQn의 인터럽트 우선순위 설정
HAL_NVIC_EnableIRQ()	NVIC에서 EXTI2_3_IRQn 허용
EXTI2_3_IRQHandler()	EXTI Line 2/3 인터럽트 진입점
HAL_GPIO_EXTI_IRQHandler()	EXTI pending bit 정리 후 Callback 호출
HAL_GPIO_EXTI_Callback()	사용자 이벤트 처리: debounce 후 LED toggle
HAL_GPIO_TogglePin()	PA5 출력 상태 반전

Polling과 Interrupt 비교

입력 처리 방식의 차이를 CPU 관점에서 설명합니다.

Polling
CPU가 주기적으로 입력 상태를 읽음

구현이 단순함
입력을 놓치지 않으려면 반복 주기가 짧아야 함
CPU가 불필요하게 입력을 계속 확인할 수 있음
delay가 길면 반응성이 나빠짐

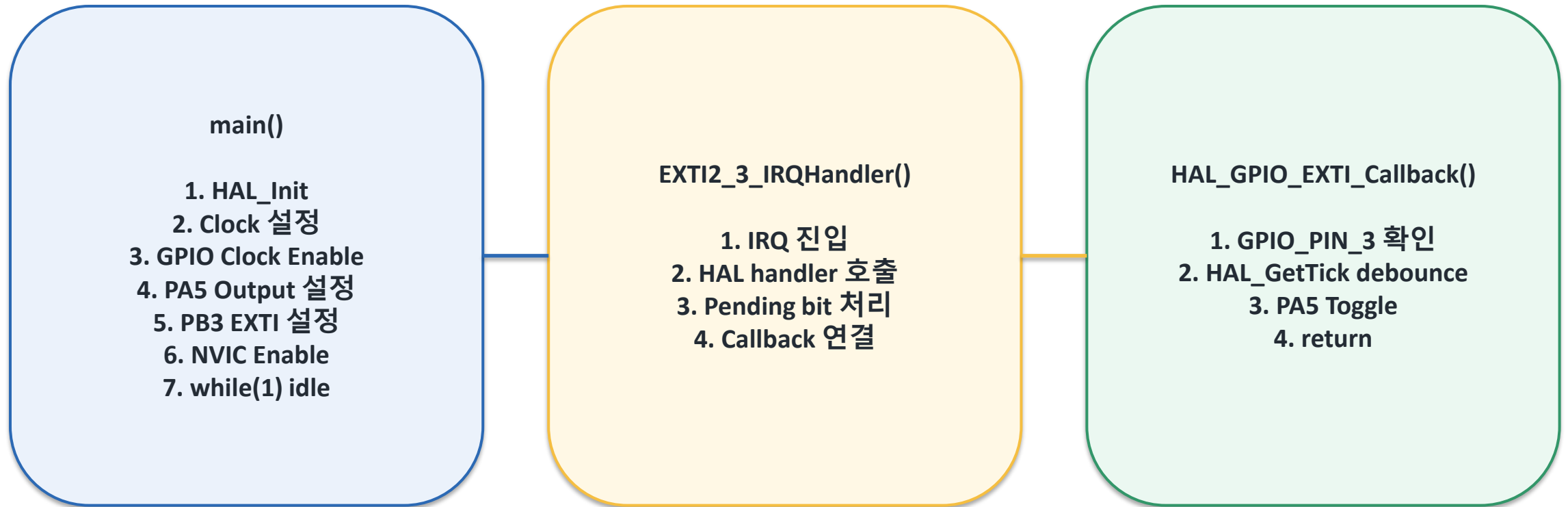
Interrupt
입력 이벤트가 CPU 실행 흐름을 깨움

이벤트가 발생할 때만 처리
main loop에서 다른 작업 또는 sleep 가능
ISR은 짧고 빠르게 작성해야 함
bounce, priority, shared variable 관리 필요

이 예제의 핵심 학습 포인트: 같은 GPIO 기능이라도 프로그램 구조는 Polling 중심에서 Event 중심으로 완전히 달라진다.

변경된 코드 구조 요약

전체 코드는 초기화 영역, IRQ Handler, Callback 영역으로 나눌 수 있습니다.



실무 권장: 복잡한 처리는 Callback에서 직접 수행하지 말고 flag만 세운 뒤 main loop/task에서 처리합니다.

PB3 입력 이벤트가 PA5 LED Toggle을 발생

① PB3를
GPIO_MODE_IT_FALLING으
로 설정

② EXTI2_3_IRQn을
NVIC에서 Enable

③ IRQ Handler가
HAL Handler 호출

④ Callback에서
Debounce 후 LED Toggle

Polling은 계속 확인하는 구조, Interrupt는 이벤트가 발생할 때만 반응하는 구조.